File permissions in Linux

Project description

You are a security professional at a large organization. You mainly work with their research team. Part of your job is to ensure users on this team are authorized with the appropriate permissions. This helps keep the system secure.

Your task is to examine existing permissions on the file system. You'll need to determine if the permissions match the authorization that should be given. If they do not match, you'll need to modify the permissions to authorize the appropriate users and remove any unauthorized access.

Check file and directory details

```
researcher2@abc4d104dce0:~$ cd projects
researcher2@abc4d104dce0:~/projects$ ls -la
total 32
drwxr-xr-x 3 researcher2 research_team 4096 Nov  1 20:16 .
drwxr-xr-x 3 researcher2 research_team 4096 Nov  1 20:42 ..
-rw--w---- 1 researcher2 research_team   46 Nov  1 20:16 .project_x.txt
drwx--x--- 2 researcher2 research_team 4096 Nov  1 20:16 drafts
-rw-rw-rw- 1 researcher2 research_team   46 Nov  1 20:16 project_k.txt
-rw-r----- 1 researcher2 research_team   46 Nov  1 20:16 project_m.txt
-rw-rw-r-- 1 researcher2 research_team   46 Nov  1 20:16 project_r.txt
-rw-rw-r-- 1 researcher2 research_team   46 Nov  1 20:16 project_t.txt
researcher2@abc4d104dce0:~/projects$
```

Here I have navigated to the projects directory using 'cd' and used 'ls -la' to check the permissions of all files within it.

Describe the permissions string

The permissions string `drwxrwxrwx` is divided into four parts. The first character indicates the file type, where `d` stands for directory. The remaining nine characters are split into three groups of three, representing permissions for the **user (owner), group and others**, respectively.

The three permission types are:

- **r (read):** Allows the user to read a file or view the contents of a directory.

- **w (write):** Allows the user to modify a file or create, delete and rename items within a directory.
- **x (execute):** Allows the user to execute a file or access and navigate into a directory.

If a hyphen (`-`) appears instead of `r`, `w` or `x`, it means that the corresponding permission has not been granted.

For example, `drwx-wxr--` means:

- **User (`rwx`):** Has read, write and execute permissions.
- **Group (`-wx`):** Has write and execute permissions but no read permission.
- **Others (`r--`):** Have read permission only.

This permission structure helps control who can access, view or modify files and directories on a Linux system.

The permissions string `drwxrwxrwx` is divided into four parts. The first character indicates the file type, where `d` stands for directory. The remaining nine characters are split into three groups of three, representing permissions for the **user (owner), group and others**, respectively.

The three permission types are:

- **r (read):** Allows the user to read a file or view the contents of a directory.
- **w (write):** Allows the user to modify a file or create, delete and rename items within a directory.
- **x (execute):** Allows the user to execute a file or access and navigate into a directory.

If a hyphen (`-`) appears instead of `r`, `w` or `x`, it means that the corresponding permission has not been granted.

For example, `drwx-wxr--` means:

- **User (`rwx`):** Has read, write and execute permissions.
- **Group (`-wx`):** Has write and execute permissions but no read permission.
- **Others (`r--`):** Have read permission only.

This permission structure helps control who can access, view or modify files and directories on a Linux system.

Change file permissions

```
researcher2@abc4d104dce0:~/projects$ chmod o-w project_k.txt
researcher2@abc4d104dce0:~/projects$ ls -l
total 20
drwx--x--- 2 researcher2 research_team 4096 Nov  1 20:16 drafts
-rw-rw-r-- 1 researcher2 research_team   46 Nov  1 20:16 project_k.txt
-rw-r----- 1 researcher2 research_team   46 Nov  1 20:16 project_m.txt
-rw-rw-r-- 1 researcher2 research_team   46 Nov  1 20:16 project_r.txt
-rw-rw-r-- 1 researcher2 research_team   46 Nov  1 20:16 project_t.txt
researcher2@abc4d104dce0:~/projects$
```

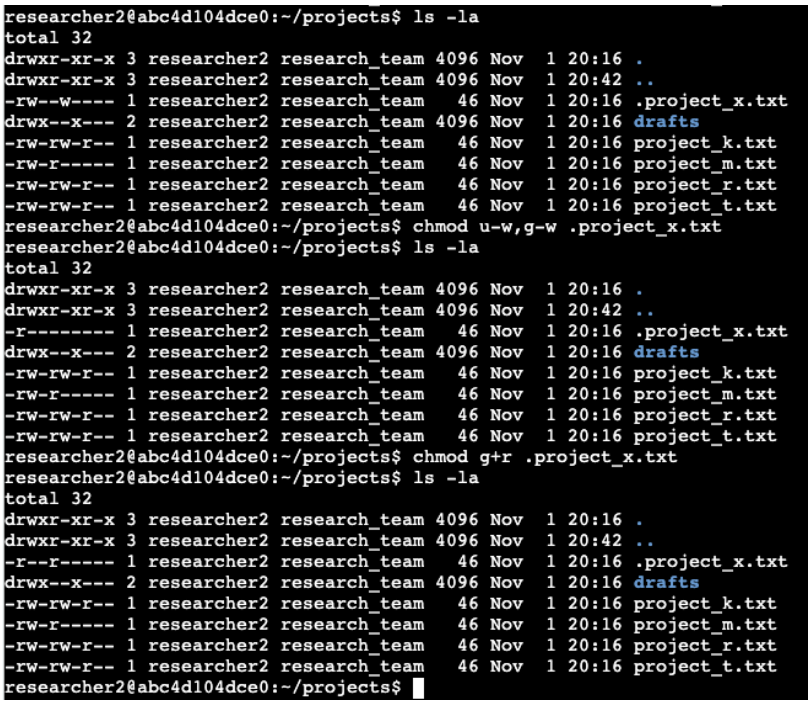
The owner type **'other'** had been given write permissions for the file `project_k.txt`, which needed to be removed. I used the command `chmod o-w project_k.txt` to do this.

The command has three parts:

1. **`chmod`** – Means change mode and is used to modify file or directory permissions.
2. **`o-w`** – The `o` represents other users, while `-w` means to remove their write permission.
3. **`project_k.txt`** – Specifies the file I want to change the permissions for.

This removed write access for other users while leaving the permissions for the owner and group unchanged.

Change file permissions on a hidden file



The file `.project_x.txt` is a hidden, archived file that should not be modified by anyone. The user and group should still have permission to read it, but neither should have write permissions.

To achieve this, I used the following commands:

1. **`chmod u-w,g-w .project_x.txt`** – Removed write permissions from both the user (`u`) and group (`g`).
2. **`chmod g+r .project_x.txt`** – Added read permissions for the group (`g`), as the group still needed to be able to read the file.

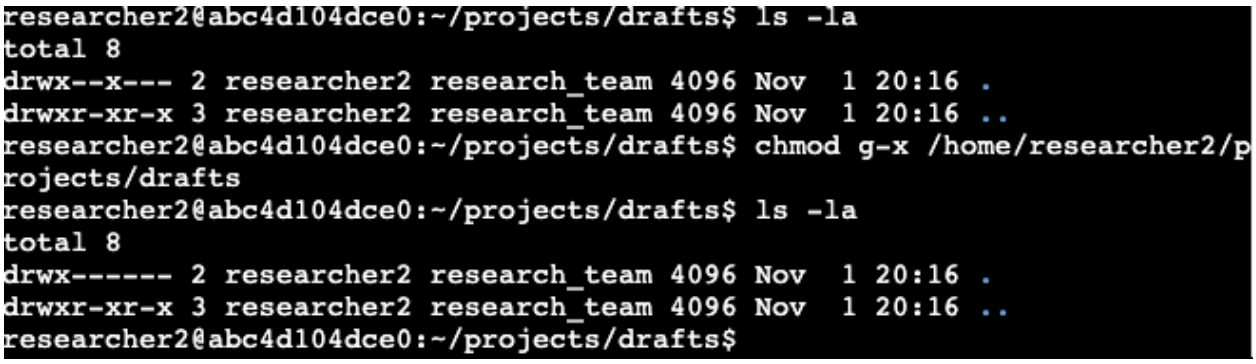
I then used `ls -la` to check the permissions and confirm that the changes had been applied.

The final permissions were `-r--r-----`, meaning:

- **User (`r--`):** Can read but cannot write or execute the file.
- **Group (`r--`):** Can read but cannot write or execute the file.
- **Others (`---`):** Have no permissions.

This ensures that the user and group can read the archived file, but nobody has write permissions.

Change directory permissions



The `drafts` directory originally had execute permissions for the group. I needed to remove this permission, so I used the command `chmod g-x /home/researcher2/projects/drafts`.

The command has three parts:

1. **`chmod`** – Means change mode and is used to modify file or directory permissions.
2. **`g-x`** – The `g` represents the group, and `-x` means to remove execute permissions.
3. **`/home/researcher2/projects/drafts`** – Specifies the directory whose permissions I want to change.

I then used `ls -la` to check that the permissions had changed from `drwx--x---` to `drwx------`.

This means the owner still has read, write and execute permissions, but the group and other users no longer have any permissions.

Summary

Linux File and Directory Permissions

As part of this exercise, I practised using Linux commands to inspect and modify file and directory permissions. I used `ls -la` to view file permissions, including hidden files, and identify which permissions were assigned to the owner, group and other users.

I used `chmod` to make changes based on specific access requirements. This included removing write permissions from other users on `project_k.txt`, restricting write access to the hidden archived file `.project_x.txt` while retaining read access for the owner and group, and removing execute permissions for the group on the `drafts` directory.

I also explored how Linux represents the current directory (`.`) and its parent directory (`..`) when listing directory contents.

This exercise helped me develop a better understanding of Linux access controls, file permissions and how to use the command line to manage access to files and directories.